

1. Atividades Práticas - Gerenciamento de Estado e Ciclo de Vida dos Componentes

Nesta atividade prática, vamos trabalhar com os conceitos de sobre o gerenciamento de Estado e Ciclo de Vida dos componentes do React considerando uma concessionária de carros.

1.1 Criando o projeto e iniciando o servidor local

Crie um projeto chamado **concessionaria**, digitando o seguinte comando no terminal:

```
npm create vite@latest concessionaria -- --template react
```

Este processo de configuração inicial do projeto leva alguns segundos. Ao terminar, o Vite repassa instruções para que você termine de instalar as dependências do seu projeto. Desta forma, digite o seguinte comando para **entrar no diretório** recém criado do nosso projeto:

```
cd concessionaria
```

Em seguida, **instale as dependências** necessárias do projeto executando no terminal o seguinte comando:

```
npm install
```

Até aqui, você criou um projeto React usando o Vite e adicionou todas as dependências ao projeto. Você pode, agora, **abrir a aplicação no editor Visual Studio Code**. Para isso, dentro do diretório do projeto digite o seguinte comando no terminal:

```
code .
```

Após a execução deste comando, o Visual Studio Code deverá abrir com a pasta raiz do seu projeto sendo acessada. Em seguida, você inicializará um servidor local e executará o projeto em seu navegador. Digite a seguinte linha de comando no terminal:

```
npm run dev
```

Ao executar esse script, você iniciará um servidor local de desenvolvimento, executará o código do projeto, iniciará um observador que detecta alterações no código e abrirá o projeto em um navegador web. Ele irá rodar a aplicação em modo desenvolvimento em <http://localhost:5173/>.

1.2 Criando uma lista de carros

Nosso objetivo é criar um componente responsável por exibir a listagem de carros, incluindo uma funcionalidade de filtragem por marca. Esses dados serão exibidos em uma tabela HTML. Para facilitar nosso exemplo, os carros estarão armazenados em memória.

Dessa forma, vamos começar modificando o arquivo **App.jsx** para incluir essa listagem dos carros disponíveis na concessionária, conforme quadro abaixo:

```
import { useState } from 'react';
// Dados de carros em memória
const carrosEmMemoria = [
  { id: 1, marca: 'Toyota', modelo: 'Corolla', ano: 2020, preco: 85000 },
  { id: 2, marca: 'Honda', modelo: 'Civic', ano: 2019, preco: 90000 },
  { id: 3, marca: 'Ford', modelo: 'Focus', ano: 2018, preco: 70000 },
  { id: 4, marca: 'Chevrolet', modelo: 'Cruze', ano: 2021, preco: 95000 },
  { id: 5, marca: 'Hyundai', modelo: 'Elantra', ano: 2022, preco: 105000 },
];
function App() {
  // Estado para armazenar os carros
  const [carros, setCarros] = useState(carrosEmMemoria);
  return (
    <>
      <h1>Listagem de Carros</h1>
    </>
  );
}
export default App;
```

Neste exemplo:

- Importamos o Hook **useState** para gerenciar o estado do **componente funcional**.
- Criamos um array **carrosEmMemoria** que contém uma lista de carros, cada um com as seguintes propriedades: **id**, **marca**, **modelo**, **ano** e **preco**.
- Usamos a variável de estado **carros** para armazenar a lista de carros. Iniciamos a variável de estado **carros** com o valor de **carrosEmMemoria**.
- Usamos a função **setCarros** para atualizar o valor da variável **carros**.
- Neste momento, o componente funcional **App** retorna um JSX simples que contém um título **<h1>** com o texto "Listagem de Carros".

1.3 Mostrando a lista de carros na tela

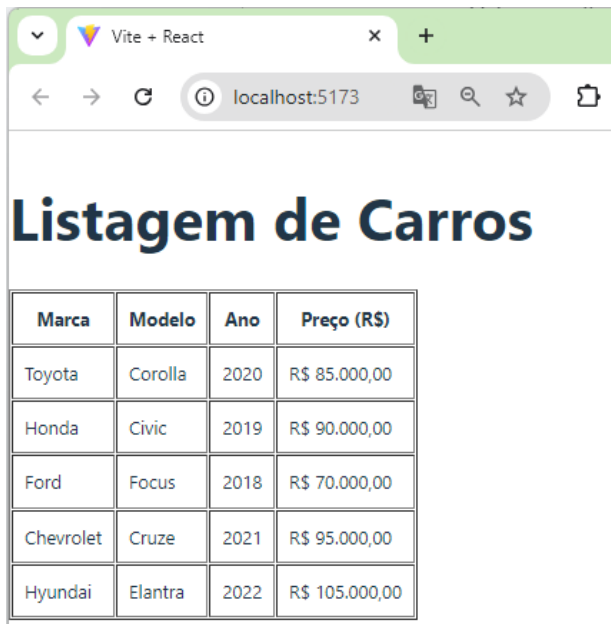
Em seguida, vamos mostrar a listagem dos carros no navegador. Para isso, precisamos percorrer as informações dos carros que estão contidas na variável de estado **carros**:

```
...
function App() {
  // Estado para armazenar os carros e a marca a ser filtrada
  const [carros, setCarros] = useState(carrosEmMemoria);
  return (
    <>
      <h1>Listagem de Carros</h1>
      <table border="1" cellPadding="10">
        <thead>
          <tr>
            <th>Marca</th>
            <th>Modelo</th>
            <th>Ano</th>
            <th>Preço (R$)</th>
          </tr>
        </thead>
        <tbody>
          {carros.length > 0 ? (
            carros.map((carro) => (
              <tr key={carro.id}>
                <td>{carro.marca}</td>
                <td>{carro.modelo}</td>
                <td>{carro.ano}</td>
                <td>
                  {carro.preco.toLocaleString("pt-BR", {
                    style: "currency",
                    currency: "BRL",
                  })}
                </td>
              </tr>
            ))
          ) : (
            <tr>
              <td colspan="4">Nenhum carro encontrado.</td>
            </tr>
          )}
        </tbody>
      </table>
    </>
  );
}
export default App;
```

Neste código:

- Renderizamos uma tabela HTML que exibe uma lista de carros, mostrando marca, modelo, ano e preço de cada carro.
- No cabeçalho da Tabela (**<thead>**), definimos os títulos das colunas da tabela (Marca, Modelo, Ano, Preço).
- No corpo da tabela (**<tbody>**) usamos o conceito de **renderização condicional**:
 - Se há carros disponíveis (**carros.length > 0**), o método **map()** é usado para iterar sobre o array de **carros** e renderizar uma linha **<tr>** para cada carro, exibindo suas informações (marca, modelo, ano e preço).
 - O preço é formatado em moeda brasileira (R\$) usando **toLocaleString**.
 - Se não há carros, é exibida uma mensagem "Nenhum carro encontrado".

Veja o resultado deste código no navegador:



Marca	Modelo	Ano	Preço (R\$)
Toyota	Corolla	2020	R\$ 85.000,00
Honda	Civic	2019	R\$ 90.000,00
Ford	Focus	2018	R\$ 70.000,00
Chevrolet	Cruze	2021	R\$ 95.000,00
Hyundai	Elantra	2022	R\$ 105.000,00

Figura 1: Listagem dos carros de uma concessionária

1.3 Criando o estado para filtrar uma marca

Neste momento, vamos nos preocupar em criar as funcionalidades necessárias para filtrar os carros por marca. Desta forma, altere o seguinte em nosso componente **App**:

```
...  
function App() {  
  ...  
  // Estado para armazenar a marca a ser filtrada  
  const [filtroMarca, setFiltroMarca] = useState("");  
  
  // Função para lidar com a mudança no campo de filtro de marca  
  const handleFiltrarMarca = (e) => {  
    const marca = e.target.value;  
    setFiltroMarca(marca);  
  
    // Filtrar carros pela marca  
    if (marca === "") {  
      setCarros(carrosEmMemoria); // Mostra todos os carros se o filtro estiver vazio  
    } else {  
      setCarros(  
        carrosEmMemoria.filter((carro) =>  
          carro.marca.toLowerCase().includes(marca.toLowerCase())  
        )  
      );  
    }  
  };  
  ...  
}
```

Neste código:

- Estamos usando uma variável de estado chamada **filtroMarca** para realizar o filtro de carros por marca. O estado inicial de **filtroMarca** é uma string vazia (""), o que significa que, inicialmente, não há filtro aplicado.
- Criamos a função **handleFiltrarMarca**, onde:
 - **e.target.value**: Acessa o valor que o usuário está digitando em um campo de entrada de texto (que ainda não adicionamos em nossa interface).
 - **setFiltroMarca(marca)**: Atualiza o estado **filtroMarca** com o valor que o usuário digitou para definir uma marca de carro.
 - Se o valor de **marca** for uma string vazia (ou seja, se o usuário não digitou nada no campo de filtro), a função restaura a lista completa de carros

- Senão, o método **filter()** é usado para criar uma nova lista de carros cujas marcas contêm a string digitada.
- No código **carro.marca.toLowerCase().includes(marca.toLowerCase())**, comparamos as marcas dos carros convertendo-as para letras minúsculas (**toLowerCase()**), para que o filtro funcione independentemente de maiúsculas ou minúsculas. A função **includes()** verifica se a marca do carro contém a string digitada no filtro.

1.4 Mostrando os carros de acordo com o filtro de marca

Vamos concluir o desenvolvimento deste exemplo exibindo os carros na tela do navegador conforme o filtro de marca fornecido pelo usuário.

```
...  
function App() {  
  ...  
  return (  
    <>  
    <h1>Listagem de Carros</h1>  
  
    <div>  
      <label htmlFor="filtroMarcaCarro">Filtrar por marca:</label>  
      <input  
        id="filtroMarcaCarro"  
        type="text"  
        value={filtroMarca}  
        onChange={handleFiltrarMarca}  
        placeholder="Digite a marca"  
      />  
    </div>  
    ...  
  </>  
  );  
}  
export default App;
```

Neste exemplo:

- Renderizamos um campo de entrada de texto (**<input>**) para permitir que o usuário filtre os carros por marca.
 - **id="filtroMarcaCarro"**: Define um identificador único para o campo de entrada, que é referenciado pelo rótulo (**label**).

- **type="text"**: Define o tipo do campo de entrada como texto.
- **value={filtroMarca}**: O valor do campo de entrada é controlado pelo estado **filtroMarca**. Isso significa que o valor exibido no campo de entrada está vinculado ao estado.
- **onChange={handleFiltrarMarca}**: Define a função **handleFiltrarMarca** como manipulador do evento **onChange**. Esse evento é disparado sempre que o usuário digita algo no campo, e a função atualiza o estado **filtroMarca** com o valor digitado.
- **placeholder="Digite a marca"**: Exibe o texto "Digite a marca" como placeholder no campo de entrada até o usuário começar a digitar.

Desta forma, quando o usuário digita uma **marca** no campo de entrada, o valor do campo é capturado pelo evento **onChange**, e a função **handleFiltrarMarca** é chamada. Assim, a função **handleFiltrarMarca** atualiza o estado **filtroMarca** com o valor digitado, o que, por sua vez, controla o valor exibido no campo e atualiza a lista de **carros** exibida no navegador.

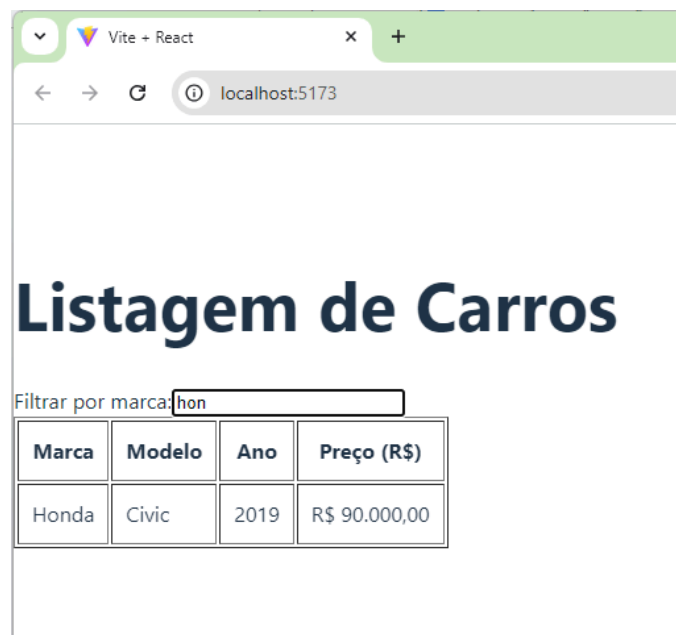


Figura 2: Filtragem dos carros de uma concessionária por marca